MPC 분산 키 생성 프로토콜 상세 분석

개요

이 문서는 TNO PET Lab의 Paillier 암호화 기반 분산 키 생성(Distributed Key Generation) 프로토콜의 동작 방식을 상세히 분석합니다. 이 프로토콜은 신뢰할 수 있는 딜러(trusted dealer) 없이 여러 참가자가 협력하여 Paillier 암호화 시스템의 공개키와 분산된 비밀키를 생성하는 방법을 제공합니다.

1. 프로토콜 초기화 단계

1.1 보안 매개변수 설정

프로토콜 시작 시 다음과 같은 보안 매개변수들이 설정됩니다:

- **corruption_threshold (t)**: 허용 가능한 최대 악의적 참가자 수
- **key_length**: 모듈러스 N의 비트 길이 (일반적으로 2048, 4096)
- **prime_threshold**: 소수성 검사 시 확인할 작은 소수들의 상한
- **correct_param_biprime**: N이 두 소수의 곱임을 보장하는 확실성 매개변수
- **stat_sec_shamir**: 정수 상의 Shamir 비밀 분산을 위한 통계적 보안 매개변수

1.2 참가자 인덱스 할당

```
async def get_indices(cls, pool: Pool) -> tuple[dict[str, int], int]:
```

각 참가자는 고유한 인덱스를 할당받습니다:

1. 각 참가자가 0~1,000,000 범위의 랜덤 숫자를 생성

2. 모든 참가자가 자신의 랜덤 숫자를 다른 참가자들에게 브로드캐스트

3. 랜덤 숫자들을 오름차순으로 정렬하여 인덱스 할당

4. 모든 랜덤 숫자의 합을 1,000,000으로 나눈 나머지를 세션 ID로 사용

1.3 Shamir 비밀 분산 스킴 초기화

두 가지 Shamir 스킴이 초기화됩니다:

- **shamir_scheme_t**: t-out-of-n 스킴 (일반적인 비밀 분산용)
- **shamir_scheme_2t**: 2t-out-of-n 스킴 (곱셈 연산 후 재구성용)

```
shamir_length = 2 * (prime_length + math.ceil(math.log2(number_of_players)))

shamir_scheme = Shamir(sympy.nextprime(2**shamir_length), number_of_players, cor
```

2. 키 생성 프로세스

2.1 소수 생성 단계

2.1.1 후보 소수 생성

각 참가자는 독립적으로 소수 p와 q의 후보를 생성합니다:

1. **작은 소수 테스트**: prime_threshold 이하의 모든 소수로 나누어 확인
2. **Miller-Rabin 소수성 테스트**: 확률적 소수성 검사 수행
3. **배치 처리**: 효율성을 위해 여러 후보를 동시에 생성 (batch_size 매개변수)

2.1.2 분산 소수성 검증

생성된 후보 소수들에 대해 분산 검증을 수행합니다:

```
async def distributed_primality_test(
candidate: int,
generators: list[int],
index: int,
pool: Pool,
party_indices: dict[str, int],
session_id: int,
) -> bool:
```

1. **Jacobi 기호 검사**: 생성자들에 대한 Jacobi 기호가 1인지 확인
2. **분산 지수 연산**: 각 참가자가 부분적인 지수 연산 수행
3. **결과 결합**: 모든 참가자의 결과를 결합하여 최종 소수성 판정

2.2 모듈러스 N 계산

2.2.1 분산 곱셈

p와 q의 곱 N = p × q를 분산 방식으로 계산합니다:

```
async def compute_modulus(
shares: Shares,
index: int,
pool: Pool,
party_indices: dict[str, int],
shamir_scheme_t: Shamir,
```

```
shamir_scheme_2t: Shamir,

session_id: int,

batch_size: int = 1,

) -> int:
```

1. **Shamir 분산**: p와 q를 각각 Shamir 비밀 분산으로 분산
2. **분산 곱셈**: 분산된 p와 q를 곱하여 분산된 N 생성
3. **재구성**: 2t+1개의 분산을 사용하여 N 재구성

2.2.2 이중 소수 검증

생성된 N이 정확히 두 소수의 곱인지 분산 방식으로 검증:

```
async def distributed_biprimality_test(

n: int,

generators: list[int],

index: int,

pool: Pool,

party_indices: dict[str, int],

session_id: int,

) -> bool:
```

2.3 비밀키 생성

2.3.1 λ(N) 계산

카마이클 함수 λ(N) = lcm(p-1, q-1)을 분산 방식으로 계산:

1. **p-1, q-1 계산**: 각 참가자가 자신의 소수에서 1을 뺌
2. **최대공약수 계산**: gcd(p-1, q-1)을 분산 방식으로 계산
3. **최소공배수 계산**: λ(N) = (p-1)(q-1) / gcd(p-1, q-1)

2.3.2 β 값 계산

β = λ(N) mod N을 계산하여 복호화에 사용할 값을 준비합니다.

2.3.3 θ 값 계산

θ = L(g^λ mod N^2)^(-1) mod N을 계산합니다 (여기서 L(x) = (x-1)/N).

3. 분산 복호화 프로세스

3.1 부분 복호화

각 참가자는 자신의 비밀키 분산을 사용하여 부분 복호화를 수행합니다:

```
def  partial_decrypt(self, ciphertext: PaillierCiphertext) -> int:
# 라그랑주 보간 계수 계산
lagrange_interpol_enumerator = mult_list(other_honest_players)
lagrange_interpol_denominator = mult_list([(j - self.player_id) for j in other_h
# 지수 계산
exp = (n_fac * lagrange_interpol_enumerator * self.share.shares[self.player_id])
# 부분 복호화 수행
partial_decryption = pow_mod(ciphertext_value, exp, self.n_square)
return partial_decryption
```

3.2 복호화 결합

모든 참가자의 부분 복호화 결과를 결합하여 최종 평문을 얻습니다:

```
def  decrypt(self, partial_dict: dict[int, int]) -> int:
# 모든 부분 복호화 결과를 곱함
combined_decryption = mult_list(partial_decryptions[:self.share.degree + 1]) % s
# 최종 메시지 추출
message = ((combined_decryption - 1) // self.n * self.theta_inv) % self.n
return message
```

4. 통신 프로토콜

4.1 메시지 교환

프로토콜은 다음과 같은 메시지 교환 패턴을 사용합니다:

1. **브로드캐스트**: 모든 참가자에게 동일한 메시지 전송
2. **점대점 통신**: 특정 참가자에게만 메시지 전송
3. **수집**: 모든 참가자로부터 메시지 수신

4.2 메시지 ID 체계

각 메시지는 고유한 ID를 가집니다:

- 세션 ID 포함
- 프로토콜 단계 식별자
- 해시값 (필요시)

예: `"distributed_decryption_session#{session_id}_hash#{encryption_hash}"`

5. 보안 고려사항

5.1 반정직 보안 모델

프로토콜은 반정직(semi-honest) 보안 모델을 가정합니다:

- 참가자들은 프로토콜을 정확히 따름
- 하지만 통신 내용을 통해 정보를 추론하려 할 수 있음

5.2 임계값 보안

- $n \geq 2t + 1$ 조건을 만족해야 함
- t개 이하의 참가자가 악의적이어도 보안성 유지
- 2t+1개 이상의 참가자가 협력해야 비밀 재구성 가능

5.3 통계적 보안

- Shamir 비밀 분산에서 통계적 보안 매개변수 사용
- 마스킹 값의 범위를 충분히 크게 설정하여 균등 분포에 가깝게 유지

6. 성능 최적화

6.1 배치 처리

- 여러 후보 소수를 동시에 처리하여 통신 오버헤드 감소
- batch_size 매개변수로 배치 크기 조절

6.2 사전 계산

- 작은 소수 목록 사전 생성
- 생성자(generator) 값들 사전 계산

6.3 병렬 처리

- 독립적인 연산들을 병렬로 수행
- 비동기 통신을 통한 대기 시간 최소화

7. 실제 사용 예시

7.1 로컬 실행

```
# 모든 참가자가 같은 Python 인스턴스에서 실행
distributed_paillier_schemes = loop.run_until_complete(
asyncio.gather(*[
DistributedPaillier.from_security_parameter(
pool, corruption_threshold, key_length,
prime_thresh, correct_param_biprime,
stat_sec_shamir, distributed=False
) for pool in local_pools
])
)
```

7.2 분산 실행

```
# 각 참가자가 별도의 Python 인스턴스에서 실행
distributed_paillier_scheme = loop.run_until_complete(
DistributedPaillier.from_security_parameter(
pool, corruption_threshold, key_length,
prime_thresh, correct_param_biprime,
stat_sec_shamir, distributed=True
)
)
```

8. 에러 처리 및 복구

8.1 통신 오류

- 네트워크 연결 실패 시 재시도 메커니즘
- 타임아웃 설정을 통한 무한 대기 방지

8.2 계산 오류

- 소수성 검사 실패 시 새로운 후보 생성
- 이중 소수 검증 실패 시 프로세스 재시작

8.3 일관성 검사

- 모든 참가자가 동일한 공개키를 가지는지 확인
- 복호화 결과의 일관성 검증

이 프로토콜은 현대적인 MPC 기법을 사용하여 안전하고 효율적인 분산 키 생성을 제공하며, 실제 응용에서 사용할 수 있는 수준의 성능과 보안성을 보장합니다.